

week-7 Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment

Code:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* Global variable */
5  int global_var = 50;
6
7  /* Static global variable */
8  static int static_global_var = 20;
9
10 /* Function - stored in Code/Text segment */
11 void my_function()
12 {
13     printf("This is a function.\n");
14 }
15
16 int main()
17 {
18     /* Local variable - Stack */
19     int stack_var = 30;
20
21     /* Static local variable - Data segment */
22     static int static_var = 40;
23
24     /* Dynamic variable - Heap */
25     int *heap_var = malloc(sizeof(int));
26
27     if (heap_var == NULL)
28     {
29         printf("Memory allocation failed.\n");
30         return 1;
31     }
32
33     *heap_var = 50;
34
35     printf("==== Linux Process Memory Addresses =====\n\n");
36
37     /* Code segment */
38     printf("Code address      : %p\n", (void *)my_function);
39
40     /* Global segment */
41     printf("Global variable    : %p\n", (void *)&global_var);
42
43     /* Static variable */
44     printf("Static variable     : %p\n", (void *)&static_var);
45
46     /* Static global variable */
47     printf("Static global address : %p\n", (void *)&static_global_var);
48
49     /* Heap */
50     printf("Heap variable        : %p\n", (void *)heap_var);
51
52     /* Stack */
53     printf("Stack variable       : %p\n", (void *)&stack_var);
54
55     free(heap_var);
56
57     return 0;
58 }

```

Output:

```

1  screen@hillelreddy@Stress-MacBook-Pro ~ % nano st.c
2  screen@hillelreddy@Stress-MacBook-Pro ~ % gcc st.c -o st
3  screen@hillelreddy@Stress-MacBook-Pro ~ % ./st
4
5  ===== Linux Process Memory Addresses =====
6
7  Code address      : 0x100000000
8  Global variable   : 0x100000000
9  Static variable   : 0x100000000
10 Static global address : 0x100000000
11 Heap variable      : 0x0000000000000000
12 Stack variable     : 0x7fff7bde49150
13 screen@hillelreddy@Stress-MacBook-Pro ~ %

```

Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Code:

```
UN PICO 5.09
File: ik.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;
static int static_var = 200;

int main()
{
    int stack_var = 300;
    int *heap_var;

    heap_var = malloc(sizeof(int));
    if (heap_var == NULL)
    {
        perror("malloc");
        return 1;
    }

    *heap_var = 400;

    printf("==== Process Memory Information =====\n");
    printf("Process ID (PID) : %d\n", getpid());

    printf("\nMemory Addresses:\n");
    printf("Global variable : %p\n", (void *)&global_var);
    printf("Static variable : %p\n", (void *)&static_var);
    printf("Heap variable : %p\n", (void *)&heap_var);
    printf("Stack variable : %p\n", (void *)&stack_var);
    printf("main() address : %p\n", (void *)&main);

    printf("\nProcess is running...\n");
    printf("Press ENTER to terminate.\n");

    getchar();

    free(heap_var);

    return 0;
}
```

Output:

```
Last login: Fri Sep 18 14:34:05 on tty000
greenikhilreddy@Stress-MacBook-Pro ~ % nano ik.c
greenikhilreddy@Stress-MacBook-Pro ~ % gcc ik.c -o ik
greenikhilreddy@Stress-MacBook-Pro ~ % ./ik
==== Process Memory Information =====
Process ID (PID) : 1026

Memory Addresses:
Global variable : 0x1026c000
Static variable : 0x1026c004
Heap variable : 0x000033c0036
Stack variable : 0x7f70a000018
main() address : 0x1026a0d0

Process is running...
Press ENTER to terminate.

greenikhilreddy@Stress-MacBook-Pro ~ %
```